

中山大學

SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab6-Pthreads 并行构造

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2025 年 5 月 18 日

1 实验目的

- 深入理解 C/C++ 程序编译、链接与构建过程
- 提高 Pthreads 多线程编程综合能力
- 并行库开发与性能验证

2 实验内容

- 基于 Pthreads 的并行计算库实现：parallel_for + 调度策略
- 动态链接库生成和使用
- 典型问题的并行化改造：矩阵乘法、热传导问题

3 实验过程

3.1 环境与工具

简要说明实验所使用的操作系统、编译器 (gcc/g++) 版本、以及 Pthreads 库。

- 操作系统: Ubuntu 22.04 (Linux 6.11.0-25-generic, x86_64)
- 编译器: GCC 11.4.0
- CPU: Intel Xeon Gold 6240R @ 2.40GHz, 24 核/48 线程 (双路共 96 逻辑核)
- Pthreads: Linux glibc 自带线程库 (NPTL), 编译时添加 `-pthread` 选项

3.2 核心函数实现 (`parallel_for`)

简要描述 `parallel_for` 函数的关键设计思路和实现。重点说明线程创建、任务划分和同步机制。

设计思路： `parallel_for` 函数模仿 OpenMP 的并行 for 循环机制，基于 Pthreads 实现。核心设计包括：

1. **持久线程池：**避免每次调用都创建/销毁线程的开销。首次调用时创建指定数量的工作线程，线程通过条件变量等待任务分配。使用代际计数器 (generation counter) 区分不同批次的并行任务，避免工作线程在 cleanup 阶段与新任务之间的竞态条件。

2. **静态调度 (Static)**：将迭代空间 $[start, end)$ 按线程数均匀分块。前 $remainder$ 个线程多分配一个迭代，其余线程各分配 $total/num_threads$ 个迭代。每个线程处理连续的迭代范围，适合每次迭代工作量相近的场景。

3. **动态调度 (Dynamic)**：使用互斥锁保护的共享计数器 `w_next_iter` 实现任务窃取。每个线程循环获取 `chunk_size` 个迭代，直到所有迭代被分配完毕。适合每次迭代工作量不均衡的场景，但额外的锁竞争带来开销。

4. **同步机制**：主线程通过 `pthread_cond_broadcast` 唤醒所有工作线程，通过 `pthread_cond_wait` 等待所有工作线程完成。工作线程完成工作后递增完成计数器，最后一个完成的线程通知主线程。

函数接口：

```
void parallel_for(int start, int end, int inc,
                  void *(*functor)(int, void*), void *arg,
                  int num_threads);

void parallel_for_sched(int start, int end, int inc,
                        void *(*functor)(int, void*), void *arg,
                        int num_threads, int schedule, int chunk_size);
```

其中 `schedule` 取 `SCHED_STATIC (0)` 或 `SCHED_DYNAMIC (1)`，`chunk_size` 指定动态调度的块大小。

3.3 动态库生成与使用

说明生成动态链接库 (.so) 的主要命令或 Makefile 规则，并简述如何在主程序 (如矩阵乘法、热传导) 中链接和调用该库。

生成动态库：

```
# 编译位置无关代码
gcc -Wall -O2 -fPIC -c parallel_for.c
# 创建动态库
gcc -shared -Wl,-soname,libparallel_for.so.1 \
    -o libparallel_for.so.1.0 parallel_for.o -pthread
# 创建符号链接
ln -sf libparallel_for.so.1.0 libparallel_for.so.1
ln -sf libparallel_for.so.1 libparallel_for.so
```

链接主程序：

```
# 编译主程序并链接自定义动态库
gcc -Wall -O2 -c heated_plate_pthreads.c
gcc -o heated_plate_pthreads heated_plate_pthreads.o \
    -L. -lparallel_for -Wl,-rpath,. -pthread -lm
```

其中 `-L.` 指定当前目录为库搜索路径，`-lparallel_for` 链接 `libparallel_for.so`，`-Wl,-rpath,.` 设置运行时库搜索路径。

3.4 应用测试 (矩阵乘法)

为验证 `parallel_for` 的正确性与效率，首先使用矩阵乘法进行测试。测试矩阵大小为 512×512，使用 4 线程静态调度。

正确性验证：将并行计算结果与串行参考结果进行逐元素比较，最大误差在机器精度范围内 ($< 10^{-6}$)，验证了 `parallel_for` 库任务划分和同步机制的正确性。矩阵乘法同时检验了 functor 接口对二维循环的适配能力——将内层循环封装为按行处理的 functor，每行完成一整列的乘加操作。

效率验证：矩阵乘法的时间复杂度为 $O(N^3)$，计算密度高（每个输出元素需 $2N$ 次浮点运算），是验证并行效率的理想负载。4 线程静态调度下，矩阵规模 $N = 512$ 时总计算量约 2.7 亿次浮点运算，并行版本在 4 线程下获得了接近线性加速的效果。该测试表明 `parallel_for` 在计算密集型任务上能够有效利用多核资源，线程创建和同步的开销相对于计算量可忽略不计。

3.5 应用测试 (热传导)

简述如何将 `parallel_for` 应用于热传导问题，替换原有的并行机制。

- **改造方法**：原 OpenMP 实现使用 `#pragma omp parallel for` 指令对热传导迭代的各阶段进行并行化。Pthreads 改造版本将每个并行循环封装为独立的 functor 函数，通过 `parallel_for` 调用。定义统一的上下文结构体 `hp_ctx` 包含网格数组指针 (w, u)、网格尺寸、均值、差值及互斥锁。

- **Functor 设计**：对于边界条件设置等一维循环，functor 接收行索引 i 或列索引 j 直接操作；对于内点计算等二维循环，functor 接收行索引 i，内部对整行的 j 循环进行处理，以减少 parallel_for 的函数指针调用开销。均值求和的归约操作通过互斥锁保护实现。

- **关键优化——diff_row 去锁**：最大差值归约的初始实现对每一行都执行 `pthread_mutex_lock/u` 更新全局最大值，每轮迭代产生约 500 次锁操作，16955 轮累计超 800 万次。实测发现该实现下 4 线程及以上几乎不加速（4 线程 8.08s vs 2 线程 8.64s），因为锁竞争随线程数增加反而恶化。优化方案为：在上下文结构体中增加 `row_diff[M]`

数组，每个 functor 将本行局部最大值直接写入 `row_diff[i]`（无锁操作，各行独立），待 `parallel_for` 返回后由主线程串行遍历 $[1, M-1]$ 取全局最大值——串行遍历 500 个元素的代价可忽略不计。该优化将 4 线程耗时从 8.08s 降至 4.72s（提升 41%），是本次实验中性能提升最大的单项改进。

- **测试设置**：网格大小 500×500，收敛精度 $\epsilon = 0.001$。线程数取 1、2、4、8、16。调度方式分别测试静态调度 (Static) 和动态调度 (Dynamic, chunk=1, 4, 16)。

4 实验结果与分析

4.1 性能测试结果

展示不同线程数和调度方式下，自定义 Pthreads 实现与原始 OpenMP 实现的性能对比。

表 1: 热传导问题性能对比 (Pthreads vs OpenMP, 网格大小: 500×500, 收敛精度 $\epsilon = 0.001$)

线程数	**调度方式**	**Pthreads** 时间 (s)	**OpenMP** 时间 (s)	**Pthreads/OpenMP** 时间比
1 (串行)	N/A	14.51	13.63	1.06
Pthreads: 静态调度 (Static)				
2	Static	8.70	6.94	1.25
4	Static	4.72	3.63	1.30
8	Static	4.54	2.11	2.15
16	Static	4.92	1.44	3.42
Pthreads: 动态调度 (Dynamic, chunk=16)				
2	Dynamic,16	8.12	6.94	1.17
4	Dynamic,16	4.97	3.63	1.37
8	Dynamic,16	5.14	2.11	2.43
16	Dynamic,16	3.93	1.44	2.73
Pthreads: 动态调度 (Dynamic, chunk=4)				
2	Dynamic,4	8.88	6.94	1.28
4	Dynamic,4	5.64	3.63	1.55
8	Dynamic,4	4.88	2.11	2.31
16	Dynamic,4	7.37	1.44	5.12
Pthreads: 动态调度 (Dynamic, chunk=1)				
2	Dynamic,1	12.65	6.94	1.82
4	Dynamic,1	10.11	3.63	2.79
8	Dynamic,1	12.87	2.11	6.09
16	Dynamic,1	13.08	1.44	9.09

所有配置（OpenMP 与 Pthreads 各调度策略/线程数）均在同一网格规模 (500×500) 和收敛精度 ($\epsilon = 0.001$) 下收敛于 16955 次迭代，均值均为 74.949900。迭代次数和最终结果完全一致，验证了 Pthreads 改造版本计算逻辑的正确性。

表 2: 加速比对比 (相对于各自单线程基线)

线程数	OpenMP	Static	Dyn,16	Dyn,4	Dyn,1
1	1.00×	1.00×	1.00×	1.00×	1.00×
2	1.96×	1.67×	1.82×	1.66×	1.19×
4	3.76×	3.08×	2.98×	2.62×	1.48×
8	6.45×	3.19×	2.88×	3.03×	1.17×
16	9.49×	2.95×	3.77×	2.01×	1.15×

表 3: 动态调度不同 chunk 大小在 4 线程下的性能对比

chunk 大小	时间 (s)	加速比	相对 Static
Static (基线)	4.72	3.08×	–
1	10.11	1.48×	慢 114%
4	5.64	2.62×	慢 19%
16	4.97	2.98×	慢 5%

4.2 结果分析与总结

1. 正确性：所有配置在相同条件下均收敛于 16955 次迭代，最终均值 74.949900 完全一致，证明 parallel_for 库的改造未引入计算错误。

2. 并行加速效果： Pthreads 实现成功实现了并行加速，但效果因线程数和调度策略差异显著：

- **静态调度**在 4 线程加速良好 (3.08×)，但 8 线程后出现严重扩展瓶颈（8 线程 3.19×，边际收益接近于零），16 线程因同步开销和 NUMA 效应反而退化为 2.95×。原因是线程池同步开销随线程数增加而增大：每次 parallel_for 调用需广播唤醒所有池中线程并等待全部回报，即使部分线程不参与计算也需同步。此外，双路 NUMA 架构下跨 socket 访存延迟进一步限制了高线程数的扩展性。

- **动态调度 chunk=16** 在 16 线程时达到 pthreads 最优加速比 (3.77×)，且超过同线程数静态调度 (2.95×)。因为较大 chunk 降低了锁获取频率，同时动态调度使得与数据在同一 socket 的” 近端” 线程能抢占更多工作，部分弥补了 NUMA 效应。

- **动态调度 chunk=1** 几乎无法加速，8 线程仅为 1.17×。每获取一行就需要一次加锁/解锁，498 行 × 3 次 parallel_for × 16955 迭代产生约 2500 万次锁操作，竞争完全抵消并行收益。

3. 与 OpenMP 对比： Pthreads 在所有配置下均慢于 OpenMP，且差距随线程数扩大 (2 线程 1.25× → 16 线程 3.42×)。主要原因：

- **编译器优化差异**： OpenMP 由编译器将循环体直接编译为内联代码，而 Pthreads 通过函数指针调用 functor，每次调用有间接跳转开销。

- **运行时开销**：OpenMP 的并行区域保持线程存活，内部多个 `#pragma omp for` 之间仅需 barrier 同步。Pthreads 每次 parallel_for 调用都是一次完整的 fork/join 周期（broadcast→work→gather），热传导每轮迭代 3 次调用，16955 轮共计约 5 万次同步周期。
- **NUMA 感知**：OpenMP 运行时可能采用 first-touch 策略在并行循环初始化时分散内存分配，而 Pthreads 版本由主线程统一 malloc，所有数据集中在一个 socket 的内存上，另一 socket 线程远程访存延迟更高。

4. 调度策略比较：

- 对于热传导这种每次迭代工作量均匀的应用，静态调度应是理论最优。但实验中动态调度 chunk=16 在 16 线程时反超静态 (3.93s vs 4.92s)，说明静态调度对 NUMA 效应更敏感。
- 增大 chunk 大小显著改善动态调度性能：4 线程下 chunk=16 仅比静态慢 5%，而 chunk=1 慢 114%。